

UM · UNIVASF Mobile:

5 dias × 4h — do protótipo ao MVP funcional

Cronograma de quarta (26/08) a domingo (30/08), em blocos de 4 horas diárias, com construção e validação assistidas por IA (Claude) e prompts prontos para cada etapa.



Marco final: segunda-feira, 31/08

Reunião com a Profª. Drª. Ana Emília de Melo Queiroz — validação da arquitetura híbrida, da stack e do recorte do MVP.

Projeto	UM · UNIVASF Mobile (TCC)
Curso	Engenharia de Computação — UNIVASF, Campus Juazeiro
Stack	Flutter (front, imprescindível) + Dart Frog + Supabase + ML Kit
Ritmo	4h/dia · 26/08 a 30/08 · IA (Claude) como par de construção
Reunião	Segunda, 31/08, com a orientadora

1. Stack técnica recomendada

Critérios: Flutter imprescindível no front; back-end simples de entregar em 5 dias, gratuito e fácil de justificar academicamente.

CAMADA	TECNOLOGIA	POR QUÊ
Mobile (front)	Flutter · Dart	Requisito do projeto. Uma codebase para Android + iOS, Material 3 pronto, hot reload acelera o sprint. O protótipo HTML aprovado serve de referência 1:1 de telas e cores.
Back-end / API	Dart Frog (alt.: Node.js/Express)	Dart Frog mantém uma linguagem só no projeto (Dart ponta a ponta) — argumento forte na banca.
Banco + Auth + Storage	Supabase (PostgreSQL)	Resolve login, isolamento por usuário (Row Level Security) e armazenamento dos prints — sem montar infraestrutura, plano gratuito.
OCR da grade	Google ML Kit (on-device)	OCR gratuito e offline no celular: extrai os códigos do mural (C4, 6B...) e cruza com a legenda. Privacidade reforçada: a imagem não sai do aparelho.
Dados públicos	Scrapers agendados + cron	Scripts (Dart ou Python) leem sites da PROAE/PROEN/PROEX/RU 1×/dia e gravam com fonte + data.
Construção/validação	Claude (IA)	Geração de telas a partir do protótipo HTML, revisão de código, testes e validação de requisitos — com os prompts da Seção 3.

💡 Justificativa pronta para a orientadora

“Flutter no front (multiplataforma, exigência do projeto), Dart também no back-end para manter uma linguagem única, Supabase resolvendo autenticação e banco sem custo, OCR on-device com ML Kit para a grade, e IA como ferramenta de construção e validação — com todos os prompts e decisões documentados no TCC.”

Arquitetura em 3 blocos

1 · App Flutter

Telas do protótipo aprovado + OCR local da grade + consumo da API. Modo visitante sem login; conta só para calendário e grade.

2 · API + Banco

API REST (Dart Frog) servindo cardápio, ônibus, editais e calendário; Postgres com isolamento por usuário; Storage para os prints.

3 · Pipeline de coleta

Scrapers agendados dos sites oficiais com fonte + data em cada registro; falha de coleta gera aviso de “possivelmente desatualizado”.

↗ Redirecionamento SIG@

Notas, histórico e matrícula: deep links com orientação de fluxo. Zero integração restrita, zero credencial armazenada.

2. Cronograma dia a dia — 4h/dia (qua → sex)

Cada dia tem 4 horas divididas em blocos com objetivo claro e fecha com algo demonstrável. Escopo enxuto: MVP funcional mínimo, não o app completo.

26

QUA

Setup + fundação do app Flutter

🕒 4h — 1h setup e tema · 2h30 telas (com Claude) · 30min GitHub/README

- Criar projeto Flutter, tema UM (preto/azul/amarelo) e navegação.
- Reproduzir 3 telas do protótipo: Início, Discente e Grade (mural + legenda), geradas com o Prompt P1.
- Repositório no GitHub com README.

✓ Entrega: app Flutter navega entre as 3 telas principais

27

QUI

Back-end de pé: API + banco + autenticação

🕒 4h — 1h Supabase · 1h30 API Dart Frog (Prompt P2) · 1h30 login no app

- Supabase: tabelas cardápio, ônibus, editais, calendário, usuários, grades.
- API com endpoints GET públicos.
- Login por e-mail (Supabase Auth) + bloqueio do calendário sem conta, como no protótipo.

✓ Entrega: app faz login e consome dados reais da API

28

SEX

Pipeline de dados públicos + telas de conteúdo

🕒 4h — 1h30 1º scraper (Prompt P3) · 1h30 telas RU/Ônibus/Editais · 1h redirects SIG@

- 1º scraper real (cardápio do RU ou editais PROEN) gravando fonte + data; cron diário.
- Telas de conteúdo consumindo a API com selo “atualizado automaticamente”.
- Telas de redirecionamento ao SIG@ (notas, histórico).

✓ Entrega: 1 fonte oficial real alimentando o app de ponta a ponta

2. Cronograma dia a dia — 4h/dia (sáb → dom)

Final de semana reservado ao diferencial do projeto (grade por upload) e à preparação da reunião.

29
SÁB

O diferencial: upload + OCR da grade

🕒 4h — 1h upload/Storage · 2h OCR com ML Kit (Prompt P4) · 1h agenda + validação com IA

- Upload do print (image_picker) ligado à conta (Storage + RLS).
- OCR on-device: extrair códigos do mural e cruzar com a legenda (padrão real do SIG@).
- Agenda semanal gerada, visível só ao titular; botão excluir. Calendário pessoal liberado após login.
- Validar o fluxo com o Prompt P5 (revisão de código + casos de teste).

✓ Entrega: print do SIG@ vira agenda organizada no app — a demo central

30
DOM

Polimento + pacote para a orientadora

🕒 4h — 1h30 correções visuais · 1h vídeo de demo · 1h30 documento + pauta (Prompt P6)

- Revisar fluxos e quebras visuais; gravar vídeo de 1–2 min do app rodando.
- Documento: RF13/RF14 (conta e OCR), RNF de privacidade, diagrama da arquitetura.
- Pauta da reunião: o que validar (arquitetura, recorte, stack) e dúvidas (escopo dos scrapers, validação com usuários).

✓ Entrega: APK/emulador demonstrável + vídeo + documento atualizado + pauta

✓ O que estará pronto na segunda (31/08)

App Flutter navegável com identidade UM · login funcional · 1 pipeline real de dados públicos · grade do SIG@ transformada em agenda por upload/OCR · redirects do SIG@ · documento com requisitos e arquitetura atualizados.

3. Prompts prontos para o Claude

Cole cada prompt no início da sessão correspondente. Anexe o protótipo HTML e os prints do SIG@ quando indicado.

P1 · Quarta — Gerar telas Flutter a partir do protótipo

Estou construindo o app "UM · UNIVASF Mobile" em Flutter (Material 3). Anexo está meu protótipo HTML aprovado. Converta as telas Início, Área do Discente e Grade de Horários para widgets Flutter, mantendo: paleta preto #1D1B20, azul #0091DA, amarelo #FFCB05; cards brancos com sombra suave e raio 20; botões em pílula com gradiente azul; barra inferior flutuante com botão central 🎓. Gere código organizado em pastas (screens/, widgets/, theme.dart) e explique cada decisão de estrutura.

P2 · Quinta — API Dart Frog + Supabase

Crie uma API REST em Dart Frog conectada ao Supabase (PostgreSQL) para um app universitário. Endpoints GET públicos: /cardapio, /onibus, /editais, /calendario. Cada registro deve retornar fonte_oficial e atualizado_em. Modele as tabelas com RLS: dados públicos legíveis por todos; tabela "grades" legível apenas pelo dono (auth.uid()). Inclua o SQL de criação e o código das rotas com tratamento de erros.

P3 · Sexta — Scraper de dados públicos

Escreva um script em Dart (ou Python) que coleta o cardápio semanal do Restaurante Universitário da UNIVASF do site oficial [URL] e grava no Supabase com campos: dia, refeição, descrição, fonte_oficial, atualizado_em. Adicione: timeout, tratamento de falha (marca registro como possivelmente desatualizado) e log. Explique como agendar execução diária (cron).

3. Prompts prontos (continuação)

Os prompts de validação (P5, P6) são os mais importantes para o TCC: documentam o uso de IA no processo.

P4 · Sábado — OCR da grade com ML Kit

No meu app Flutter, implemente o fluxo: (1) usuário envia print da "Grade de Horário" do SIG@ (image_picker); (2) google_mlkit_text_recognition extrai o texto on-device; (3) o parser identifica códigos de turma (ex.: C4, 6B, X1) no mural e cruza com a legenda do rodapé (formato: CCMP0000 · NOME DA DISCIPLINA · Turma · Sala · Situação); (4) gera a agenda semanal Seg-Sáb em faixas 08-10, 10-12, 14-16, 16-18. Anexo: exemplo real do print. Inclua tela de revisão para o usuário confirmar/corrigir antes de salvar.

P5 · Sábado — Validação com IA (revisão + testes)

Atue como revisor sênior. Revise o código Flutter/Dart anexo do módulo de grade: (1) encontre bugs e riscos de privacidade (a imagem e a grade só podem ser acessíveis ao dono); (2) proponha casos de teste para o parser de OCR, incluindo prints com turmas em sábado, disciplinas de 4 horas e legenda com nomes longos; (3) sugira melhorias de UX para o fluxo upload → revisão → agenda. Responda em checklist objetivo.

P6 · Domingo — Atualizar o documento do TCC

Com base neste resumo do que foi implementado [colar resumo da semana], reescreva para o meu TCC: (1) os requisitos RF13 (conta de usuário/privacidade) e RF14 (OCR da grade) no padrão "O sistema deve..."; (2) um RNF novo sobre não armazenar credenciais do SIG@; (3) um parágrafo de justificativa da arquitetura híbrida (dados públicos automatizados + restritos redirecionados ao SIG@), citando a LGPD. Texto acadêmico em português, norma ABNT.



Boas práticas com IA neste sprint

- Sempre anexe o protótipo HTML/prints — contexto vale mais que descrição.
- Peça explicação junto do código: você precisa defender cada decisão na banca.
- Registre os prompts e respostas-chave num apêndice — vira evidência metodológica do uso de IA no TCC.
- IA revisa e acelera; a decisão final e o entendimento são seus.
- Revise visualmente cada tela gerada antes de seguir para a próxima.

4. Riscos, planos B e pauta da reunião

Antecipe os pontos frágeis do sprint e chegue à reunião com perguntas objetivas.

Riscos e planos B

RISCO	PLANO B
OCR não ler bem o mural do SIG@ em 1 dia	Fallback embutido: tela de revisão/correção manual após o OCR (já prevista no P4). Se travar total, cadastro manual da grade como v1.
Scraper quebrar ou site oficial fora do ar	Cada fonte é independente; falha gera aviso “possivelmente desatualizado” + link oficial. Uma fonte funcionando já demonstra o pipeline.
Supabase/RLS consumir tempo demais	Reduzir para login simples + grade local no aparelho (SQLite) na demo; migrar depois.
Escopo estourar nas 4h diárias	Cortar nesta ordem: push → scrapers extras → calendário avançado. Nunca cortar: login, 1 pipeline real e grade por upload.

Pauta objetiva para segunda-feira

- **Validar a arquitetura híbrida:** público automatizado + restrito redirecionado ao SIG@ + grade por upload.
- **Validar a stack:** Flutter + Dart Frog + Supabase + ML Kit — gratuita e justificável academicamente.
- **Demonstrar:** vídeo/APK do fluxo completo, com ênfase no print virando agenda — o diferencial “palpável” frente ao trabalho anterior.
- **Mencionar o método:** construção e validação assistidas por IA (Claude), com prompts documentados — tema valorizado em TCC de Engenharia de Computação.
- **Alinhar a escrita:** confirmar RF13/RF14, RNF de privacidade e telas substituídas no capítulo do MVP.
- **Perguntar:** formato da validação com usuários e prazo da próxima entrega parcial.

Checklist físico para a reunião

Celular com o app instalado (ou emulador pronto) · vídeo de backup · este plano impresso · plano de reestruturação anterior · print do SIG@ usado no teste de OCR · registro dos prompts utilizados.

Regra do sprint

4 horas por dia, cada dia com algo demonstrável. Travou por mais de 30 minutos? Consulte o Claude com o prompt da etapa. Travou por 2 horas? Ative o plano B e siga — segunda é sobre mostrar direção e funcionamento, não perfeição.